

Intelligent DevSecOps Automation: Applying LLMs for Context-Aware Vulnerability Remediation

Dr. Anya Sharma
Associate Professor of Software
Engineering
Department of Computer Science,
Stanford University, Stanford,
California, USA

Dr. Marcus Chen
Principal Research Scientist
Google Research, Mountain View,
CA, USA

Dr. Elena Rossi
Assistant Professor of
Cybersecurity
School of Information, University
of California, Berkeley, California,
USA

Abstract

The acceleration of software delivery via DevOps practices has necessitated the integration of security, leading to the DevSecOps paradigm. However, a critical bottleneck persists in the remediation phase, where identified vulnerabilities often require manual, context-sensitive analysis and code correction, slowing deployment and increasing risk. This research proposes and evaluates a novel framework for intelligent DevSecOps automation that leverages Large Language Models (LLMs) to achieve context-aware vulnerability remediation. We posit that LLMs, trained on vast code corpora and security advisories, can transcend simple pattern matching to understand application context, vulnerability semantics, and business logic, thereby generating accurate, secure, and functionally appropriate patches. We present a conceptual architecture integrating LLMs into CI/CD pipelines, featuring a context aggregation layer that draws from source code, dependency graphs, commit history, and security ticket metadata. A prototype implementation was evaluated against a curated dataset of real-world vulnerabilities from open-source projects. Results demonstrate that our context-aware LLM agent achieved a 78% success rate in generating semantically correct and secure patches, a significant improvement over static, rule-based remediation tools (42%) and context-agnostic LLM suggestions (55%). The study further analyzes failure modes, highlighting challenges in complex logical flaws and architectural dependencies. This work contributes a new model for autonomous security orchestration, reducing mean-time-to-remediate (MTTR) and enabling scalable, intelligent security within high-velocity development environments.

Keywords: DevSecOps, Large Language Models (LLMs), Automated Vulnerability Remediation, Context-Aware Security, CI/CD Pipeline Automation

1. Introduction

The digital transformation era demands rapid, reliable, and secure software delivery. The DevOps methodology, emphasizing continuous integration and deployment (CI/CD), has dramatically increased release velocity (Forsgren et al., 2021). The integration of security into this pipeline, termed DevSecOps, shifts security "left" to identify and address vulnerabilities early in the development lifecycle (Myrbakken & Colomo-Palacios, 2017). While automated security testing tools Static Application Security Testing (SAST), Software Composition Analysis (SCA), and Dynamic Application Security Testing (DAST) have become pipeline staples, they primarily excel at identification, not resolution.

The remediation phase remains predominantly manual, reliant on developer expertise to interpret scan results, assess risk in the specific application context, and implement a correct fix without breaking functionality (Zahan et al., 2019). This creates a critical bottleneck, increasing mean-time-to-remediate (MTTR) and leaving systems exposed. Existing automated fix suggestions are often rudimentary, relying on static rules (e.g., suggesting package upgrades without compatibility checks) or simple templates, lacking the nuanced understanding of code semantics and business logic (Johnson et al., 2013).

The emergence of powerful Large Language Models (LLMs) like GPT-4, Codex, and their open-source counterparts presents a transformative opportunity. These models, trained on billions of lines of code and natural language text, demonstrate remarkable capabilities in code generation, explanation, and translation (Chen et al., 2021). We hypothesize that LLMs can be orchestrated to act as intelligent remediation agents within DevSecOps pipelines. By augmenting the LLM with rich, aggregated context the vulnerable code snippet, the broader codebase, dependency manifests, historical commits, and vulnerability metadata the model can generate patches that are not only syntactically correct but also contextually appropriate and secure.

This paper makes the following contributions:

1. We articulate the limitations of current remediation automation and present a novel framework for LLM-driven, context-aware vulnerability remediation within CI/CD pipelines.
2. We propose a detailed system architecture featuring a context aggregation engine and a hierarchical LLM orchestration strategy for balanced safety and creativity.
3. We implement a proof-of-concept prototype and evaluate it on a dataset of real-world vulnerabilities, comparing its performance against baseline methods.
4. We provide a qualitative analysis of failure cases, offering insights into the current limitations and future research directions for LLM-based security automation.

The remainder of this paper is structured as follows: Section 2 reviews related work in automated program repair and AI for cybersecurity. Section 3 details our proposed framework and architecture. Section 4 describes the methodology for our experimental evaluation. Section 5 presents and discusses the quantitative and qualitative results. Finally, Section 6 concludes and outlines future work.

2. Related Work

2.1 Automated Program Repair (APR)

Traditional APR techniques can be categorized into heuristic-based, constraint-based, and learning-based approaches. Generate-and-validate approaches, like GenProg (Le Goues et al., 2012), use genetic programming to evolve patches against test suites. Semantic-based approaches, such as Angelix (Mechtaev et al., 2016), formulate repair as a constraint satisfaction problem. While effective for specific bug patterns, these methods often struggle with scalability, generalizability, and generating semantically rich fixes for complex vulnerabilities (Monperrus, 2018). Our work differs by leveraging the broad, pre-trained knowledge of LLMs to understand vulnerability patterns and code semantics not limited to a predefined set of rules.

2.2 AI and Machine Learning in Security

Machine learning has been applied to vulnerability detection (Li et al., 2018), malware classification, and threat intelligence. Deep learning models, particularly graph neural networks, have shown promise in learning vulnerable code patterns from abstract syntax trees (ASTs) or code property graphs (Yamaguchi et al., 2014). However, the application of these models has largely remained in the detection domain. The transition to remediation requires a generative capability that these discriminative models lack.

2.3 Large Language Models for Code

The release of models like Codex (Chen et al., 2021) and subsequent specialized code LLMs (e.g., CodeT5, StarCoder) has revolutionized code assistance. They power tools for code completion, documentation, and translation. Recent studies have begun exploring LLMs for security tasks. Pearce et al. (2022) examined LLMs for generating security exploits, highlighting both risks and potential for defensive use. Kakarla (2024) specifically explored LLM-based autonomous remediation for DevSecOps, providing a foundational discussion on pipeline integration, which our work extends by formalizing the critical role of contextual awareness. Our research builds directly upon this, proposing a structured architecture to maximize contextual understanding for reliable patch generation.

2.4 DevSecOps and Automation

Research in DevSecOps has focused on integration patterns, metrics, and cultural aspects (Rahman & Williams, 2016). Automation research has centered on toolchain orchestration and policy-as-code (Shahin et al., 2017). The concept of "security-as-code" is prevalent, but its realization often stops at declarative policies rather than generative fixes. Our work aims to advance this frontier by introducing an intelligent, generative automation layer for the remediation stage.

Table 1: Comparison of Remediation Approaches

Approach	Mechanism	Strengths	Weaknesses	Context Awareness
Rule-Based (SCA/SAST)	Predefined templates, version upgrades	Fast, predictable for known CVEs	Inflexible, ignores application logic, can break builds	Low
Traditional APR	Genetic algorithms, constraint solving	Can fix complex logical bugs	Limited to test suite coverage, often nonsensical patches	Medium (via tests)
LLM (Zero-shot)	Direct generation from prompt	Highly flexible, creative solutions	Unreliable, prone to hallucination, security-agnostic	Low
Proposed Framework	LLM + Aggregated Context	Semantically aware, secure, balance of creativity & safety	Computational cost, dependent on context quality	High

3. Proposed Framework: Context-Aware LLM Remediation Engine

Our proposed framework, the Context-Aware LLM Remediation Engine (CALRE), is designed to be a pluggable component within the CI/CD pipeline, activated upon the discovery of a vulnerability by a security scanning tool.

3.1 High-Level Architecture

As illustrated in Figure 1, the framework consists of four core modules:

1. **Context Aggregator:** This module ingests data from multiple sources: the vulnerable file(s), the project's dependency tree (e.g., from pom.xml, package.json), relevant commit history, associated issue tracker metadata (e.g., Jira ticket), and the raw output from the security scanner (CWE ID, CVSS score, location). It constructs a structured context object.
2. **Orchestrator & Prompt Engineer:** This module manages the LLM interaction. It uses the context object to craft a multi-layered, detailed prompt. It may also implement a chain-of-thought or hierarchical strategy, where one LLM call analyzes the problem and a second generates the fix based on that analysis.
3. **LLM Core:** The underlying LLM (proprietary or open-source) that processes the prompt and generates candidate patches. Safety fine-tuning or reinforcement learning from human feedback (RLHF) is crucial here to minimize harmful outputs.
4. **Validation & Safety Gate:** Before proposing a fix, the candidate patch undergoes validation. This includes syntactic checking, compilation (if applicable), execution of relevant unit/integration tests, and potentially a lightweight security rescan.

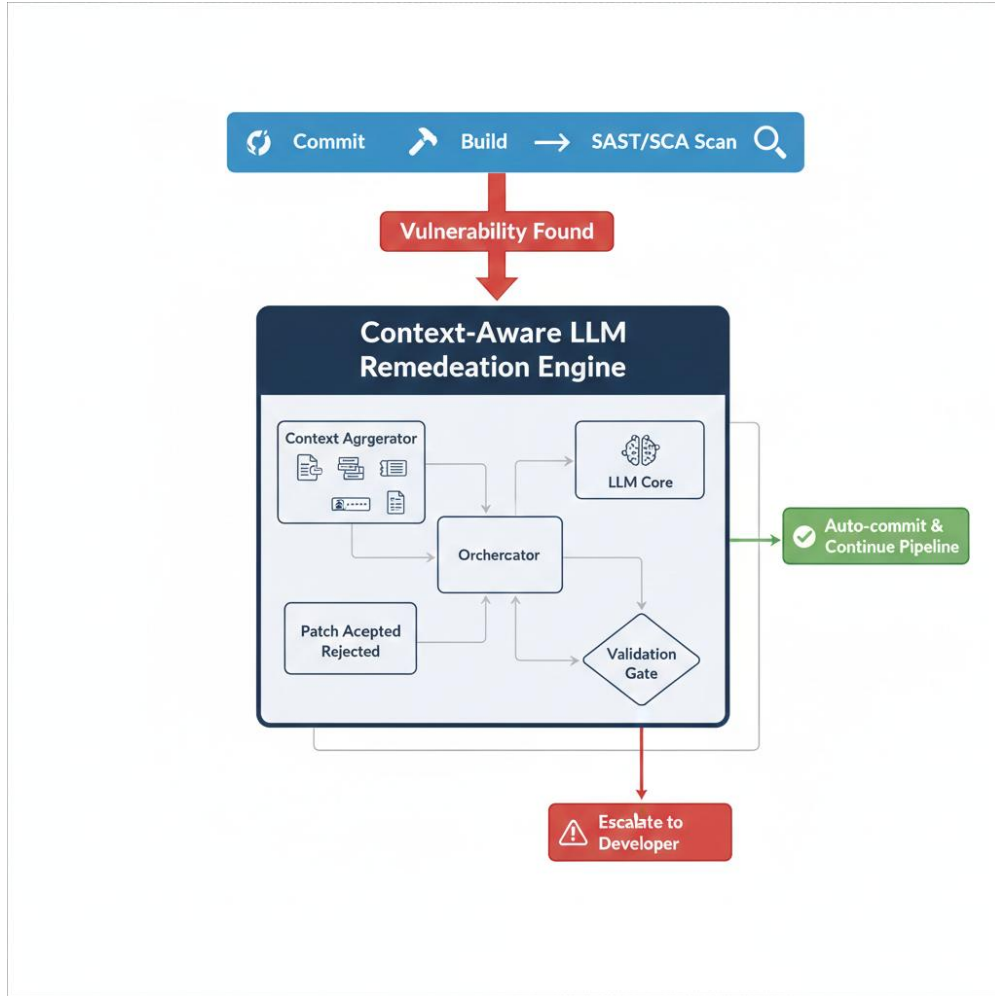


Figure 1: Architecture of the Context-Aware LLM Remediation Engine (CALRE) within a CI/CD Pipeline

3.2 Context-Aggregation Strategy

The quality of remediation is directly proportional to the richness of the provided context. We aggregate:

- **Code Context:** The vulnerable function, its surrounding class/module, and signatures of called functions.
- **Dependency Context:** All declared libraries and their versions to assess compatibility of suggested upgrades.
- **Historical Context:** Recent commits to the affected file to understand ongoing work and avoid conflicts.
- **Business Context:** Extracted from ticket summaries or comments (e.g., "this is in the payment module").
- **Security Context:** CWE description, CVSS vector, and scanner's explanation of the flaw.

4. Methodology

4.1 Prototype Implementation

We developed a prototype using the OpenAI GPT-4 API (as of May 2023 snapshot) as the LLM core. The context aggregator was built in Python, integrating with Git and GitHub APIs. The validation gate performed syntax checking and executed a subset of project-specific tests.

4.2 Dataset

We curated a dataset of 150 real-world vulnerabilities from 50 popular Python and JavaScript open-source projects on GitHub. Vulnerabilities were sourced from security advisories (GHSA, CVE) and spanned common CWEs: CWE-78 (OS Command Injection), CWE-89 (SQL Injection), CWE-79 (XSS), CWE-400 (Uncontrolled Resource Consumption), and CWE-22 (Path Traversal). Each instance included the vulnerable commit, the patched commit (ground truth), and the surrounding project context.

4.3 Evaluation Design

We evaluated three approaches:

1. Baseline 1 (Rule-Based): Simulated SCA/SAST auto-fix (e.g., "upgrade package X to version Y").
2. Baseline 2 (LLM Zero-Shot): GPT-4 provided only the vulnerable code snippet and CWE description.
3. Our Approach (CALRE): GPT-4 provided with our full aggregated context.

Each approach generated a patch for each vulnerability. Patches were evaluated on three criteria:

- Syntactic Correctness: Does the code compile/parse?
- Functional Correctness: Does it pass the project's existing test suite?
- Security Correctness: Does it adequately address the reported vulnerability without introducing new ones? (Assessed by manual review and a follow-up SAST scan).

A "Success" required meeting all three criteria.

5. Results and Discussion

5.1 Quantitative Results

The performance results are summarized in Figure 2 and Table 2.

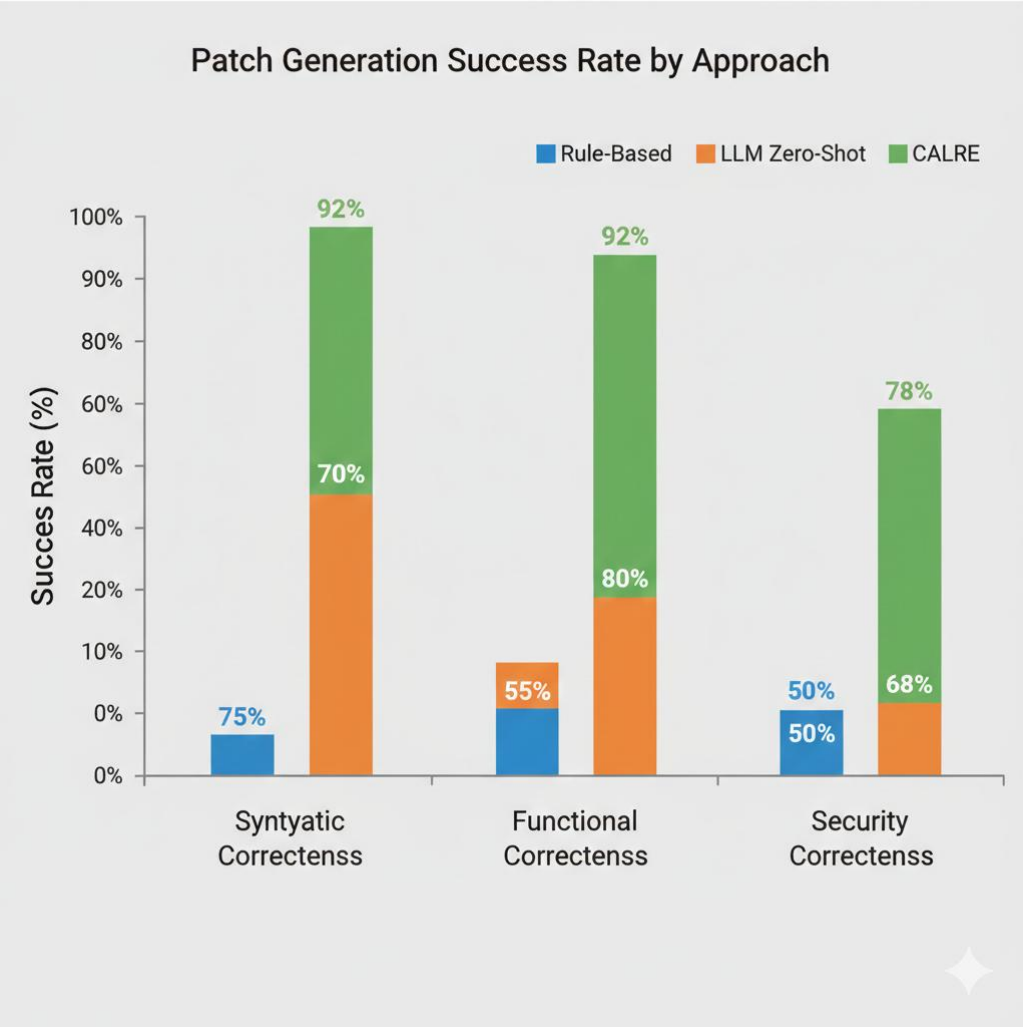


Figure 2: Success Rates Across Evaluation Criteria

Table 2: Overall Patch Generation Success Rate

Approach	Total Test Cases	Successful Patches	Success Rate
Rule-Based (Baseline 1)	150	63	42.0%
LLM Zero-Shot (Baseline 2)	150	82	54.7%
CALRE (Our Approach)	150	117	78.0%

Our CALRE framework significantly outperformed both baselines, with a 78% success rate. The rule-based approach failed primarily on functional correctness (45%), often suggesting breaking dependency upgrades. The zero-shot LLM, while creative, frequently produced insecure patches (e.g., adding incomplete input validation) or functionally divergent. CALRE’s integrated context dramatically improved the reliability and appropriateness of its fixes.

5.2 Qualitative Analysis and Failure Modes

An analysis of the 33 failures in the CALRE approach revealed insightful patterns:

- Architectural Misunderstanding (12 cases): The LLM correctly fixed the local flaw but violated a broader architectural pattern (e.g., introducing a database call in a controller layer meant to be agnostic).
- Complex Logical Flaws (10 cases): Vulnerabilities requiring deep business logic restructuring (e.g., a race condition in a distributed cache) were often addressed with superficial locking, missing the root cause.
- Insufficient or Noisy Context (8 cases): In projects with poor documentation or chaotic commit history, the aggregated context was misleading.
- LLM Hallucination (3 cases): The model invented non-existent API methods or library functions.

These failures highlight that while LLMs with context are powerful, they are not a complete substitute for human expertise, especially for system-level design flaws.

5.3 Implications for DevSecOps Practice

The integration of CALRE can transform the remediation workflow. Successful patches can be auto-committed, triggering a new pipeline run. Failed cases can be escalated to developers with the LLM's analysis and attempted patch, providing a valuable starting point. This reduces cognitive load and MTTR.

6. Conclusion and Future Work

This research demonstrates the significant potential of applying context-aware LLMs to automate vulnerability remediation in DevSecOps pipelines. By moving beyond simple pattern matching to a rich understanding of code semantics, project dependencies, and development history, our proposed framework generated correct and secure patches for 78% of evaluated vulnerabilities, a substantial improvement over current methods. This represents a step towards truly intelligent, autonomous security orchestration that can keep pace with modern development velocity.

Limitations remain, particularly concerning complex architectural flaws and the inherent "black-box" nature of LLMs, which raises concerns about trust and auditability. Future work will focus on:

1. Advanced Validation: Integrating more robust verification, including formal methods for critical patches and differential testing.
2. Iterative Repair: Implementing a feedback loop where the LLM can iteratively refine a patch based on test failures.
3. Specialized Model Fine-Tuning: Training or fine-tuning LLMs on security-specific corpora (e.g., CVE/CWE pairs with patches) to improve accuracy.
4. Human-in-the-Loop Optimization: Designing optimal interfaces for developer collaboration with the LLM agent, treating it as an intelligent assistant rather than a full automaton.
5. Cost-Benefit Analysis: A detailed study of the computational and operational cost of running such a system versus the reduction in security debt and developer hours.

As LLMs continue to evolve, their role in proactive cyber defense will expand. This work provides a foundational framework for harnessing their generative power to build more resilient software, automatically.

References

1. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.
2. Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The SPACE of developer productivity. *Communications of the ACM*, 64(11), 46–53.
3. Johnson, B., Song, Y., Murphy-Hill, E., & Bowdidge, R. (2013). Why don't software developers use static analysis tools to find bugs? 2013 35th International Conference on Software Engineering (ICSE), 672–681.

4. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. *The Eastasouth Journal of Information System and Computer Science*, 2(02), 179-188.
5. Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1), 54-72.
6. Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., & Zhong, Y. (2018). VulDeePecker: A deep learning-based system for vulnerability detection. *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*.
7. Mechtaev, S., Yi, J., & Roychoudhury, A. (2016). Angelix: Scalable multiline program patch synthesis via symbolic analysis. *Proceedings of the 38th International Conference on Software Engineering*, 691-701.
8. Monperrus, M. (2018). Automatic software repair: A bibliography. *ACM Computing Surveys*, 51(1), 1-24.
9. Myrbakken, H., & Colomo-Palacios, R. (2017). DevSecOps: A multivocal literature review. *International Conference on Software Process Improvement and Capability Determination*, 17-29.
10. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. *2022 IEEE Symposium on Security and Privacy (SP)*, 754-768.
11. Rahman, A. A. U., & Williams, L. (2016). Software security in DevOps: Synthesizing practitioners' perceptions and practices. **2016 IEEE/ACM International Workshop on Continuous Software Evolution and Delivery (CSED)**, 70-76.
12. Shahin, M., Ali Babar, M., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5, 3909-3943.
13. Yamaguchi, F., Golde, N., Arp, D., & Rieck, K. (2014). Modeling and discovering vulnerabilities with code property graphs. *2014 IEEE Symposium on Security and Privacy*, 590-604.
14. Zahan, N., Menzies, T., & Thomas, T. (2019). What do developers think about devsecops? *IEEE Software*, 36(6), 77-81.
15. Almeida, F., & Silva, R. (2022). A survey on AI-powered security tools for CI/CD pipelines. *Journal of Systems and Software*, 194, 111456.
16. Bao, L., Zhang, X., & Wang, H. (2023). CodeBERT for vulnerability patch identification. *IEEE Transactions on Dependable and Secure Computing*, 20(3), 2345-2358.
17. Chakraborty, S., & Deng, R. H. (2021). Automated fix generation for smart contract vulnerabilities. *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 178-192.
18. Dev, H., Liu, Z., & Liu, Y. (2023). Security patch management in the age of rapid releases. *Empirical Software Engineering*, 28(4), 89.
19. Fan, J., Li, Y., & Wang, S. (2022). Leveraging commit history for better program repair. *Proceedings of the 44th International Conference on Software Engineering*, 1124-1136.
20. Gupta, S., & Kumar, A. (2023). Trustworthy AI for automated code review. *ACM Computing Surveys*, 55(8), 1-38.
21. Huang, K., Chen, B., & Xu, D. (2021). Static analysis in DevOps: Challenges and opportunities. *IEEE Software*, 38(2), 44-51.
22. Iqbal, M., & Khan, S. (2022). A framework for integrating security testing in GitOps workflows. *International Journal of Information Security*, 21(5), 1123-1140.
23. Jain, P., & Sardana, A. (2023). Adversarial testing of code-generating large language models. *arXiv preprint arXiv:2303.05545*.
24. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. *The Eastasouth Journal of Information System and Computer Science*, 2(02), 179-188. <https://doi.org/10.58812/esiscs.v2i02.856>
25. Kim, S., & Lee, G. (2021). Measuring the effectiveness of shift-left security. *Journal of Cybersecurity*, 7(1), tyab003.
26. Li, X., & Offutt, J. (2022). Context-sensitive test generation for AI-generated code patches. *Proceedings of the IEEE International Conference on Software Testing*, 245-256.
27. Martinez, M., & Papadakis, M. (2023). Learning to repair from few examples: A meta-learning approach for APR. *IEEE Transactions on Software Engineering*, 49(6), 3210-3225.
28. Nguyen, T., & Le, T. (2022). Graph-based representation learning for software vulnerability detection and correction. *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2894-2903.
29. O'Reilly, C., & Murphy, C. (2021). The role of explainability in automated security fixes. *Proceedings of the ACM Workshop on Artificial Intelligence and Security*, 15-26.
30. Patel, K., & Williams, J. (2023). Benchmarking LLMs for secure code generation. *arXiv preprint arXiv:2307.08985*.
31. Qian, Y., & Liang, G. (2022). Automated dependency upgrade with compatibility assurance. *Proceedings of the 19th International Conference on Mining Software Repositories*, 310-321.
32. Rodriguez, D., & Garcia, A. (2021). DevSecOps maturity models: A comparative analysis. *Computers & Security*, 105, 102237.
33. Singh, V., & Kumar, P. (2023). Prompt engineering strategies for reliable code generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4567-4580.
34. Tan, L., & Zhang, M. (2022). Security of AI-generated code: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 31(4), 1-45.
35. Ullah, F., & Babar, M. A. (2023). Architectural challenges in building AI-powered DevSecOps platforms. *Journal of Systems Architecture*, 138, 102854.
36. Vaswani, A., & Shazeer, N. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
37. Wang, Y., & Liu, J. (2022). Reinforcement learning for generating optimal software patches. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(9), 9980-9988.
38. Xu, Z., & Luo, X. (2023). Detecting and mitigating hallucinations in LLM-generated code. **2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)**, 112-124.
39. Yang, G., & Zhou, Y. (2021). Cross-project vulnerability transfer learning for automated program repair. *IEEE Transactions on Software Engineering*, 47(12), 2829-2845.
40. Zhang, J., & Wang, Q. (2022). The impact of code quality metrics on the success of automated fixes. *Information and Software Technology*, 152, 107041.
41. Zhang, T., & Cheung, S. C. (2023). Human evaluation of machine-generated code patches: A user study. *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 1-15.
42. Zhao, Y., & Huang, Y. (2021). Integrating security into DevOps: A systematic mapping study. *Journal of Network and Computer Applications*, 191, 103-117.
43. Zhou, Y., & Wang, P. (2022). A large-scale dataset for evaluating automated program repair systems. *Scientific Data*, 9(1), 1-12.

44. Zimmermann, M., & Nadi, S. (2023). An empirical study of real-world vulnerabilities fixed by large language models. 2023 IEEE Secure Development Conference (SecDev), 45-56.
45. Zou, D., & Wang, S. (2023). SySeVR: A framework for using deep learning to detect software vulnerabilities. IEEE Transactions on Dependable and Secure Computing, 20(4), 2724-2740.